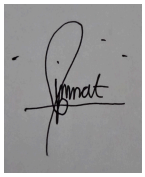
Laporan Tugas Kecil 2
Strategi Algoritma

Teknik Informatika 2026
Institut Teknologi Bandung

Anggota :

- **Nathan Adhika Santosa - 13524041**
- **Faris Wirakusuma Triawan - 13524130**

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Nathan Adhika Santosa

Faris Wirakusuma Triawan

Pseudocode Kode

geometry.hpp

```
STRUCT Vector3
   x, y, z

   FUNCTION operator-(o)
      RETURN (x - o.x, y - o.y, z - o.z)

   FUNCTION operator+(o)
      RETURN (x + o.x, y + o.y, z + o.z)

   FUNCTION operator*(scalar)
      RETURN (x * scalar, y * scalar, z * scalar)

   FUNCTION cross(o)
      RETURN (
         y * o.z - z * o.y,
         z * o.x - x * o.z,
         x * o.y - y * o.x
      )

   FUNCTION dot(o)
      RETURN x * o.x + y * o.y + z * o.z
END STRUCT


STRUCT Triangle
   v0, v1, v2
END STRUCT


STRUCT AABB
   min, max

   FUNCTION getCenter()
      RETURN (
         (min.x + max.x) / 2,
         (min.y + max.y) / 2,
         (min.z + max.z) / 2
      )

   FUNCTION getHalfExtents()
      RETURN (
         (max.x - min.x) / 2,
         (max.y - min.y) / 2,
```

```
        (max.z - min.z) / 2
    )
END STRUCT


FUNCTION testAxis(axis, v0, v1, v2, extents)
   p0 ← dot(v0, axis)
   p1 ← dot(v1, axis)
   p2 ← dot(v2, axis)

   r ← extents.x * abs(axis.x)
     + extents.y * abs(axis.y)
     + extents.z * abs(axis.z)

   epsilon ← 1e-7
   r ← r + epsilon

   minP ← minimum(p0, p1, p2)
   maxP ← maximum(p0, p1, p2)

   IF maxP < -r OR minP > r THEN
      RETURN false
   ENDIF

   RETURN true
END FUNCTION


FUNCTION checkIntersect(box, triangle)
   c ← box.getCenter()
   e ← box.getHalfExtents()

   v0 ← triangle.v0 - c
   v1 ← triangle.v1 - c
   v2 ← triangle.v2 - c

   f0 ← v1 - v0
   f1 ← v2 - v1
   f2 ← v0 - v2

   axes ← [
      (0, -f0.z, f0.y), (0, -f1.z, f1.y), (0, -f2.z, f2.y),
      (f0.z, 0, -f0.x), (f1.z, 0, -f1.x), (f2.z, 0, -f2.x),
      (-f0.y, f0.x, 0), (-f1.y, f1.x, 0), (-f2.y, f2.x, 0)
   ]

   FOR each axis IN axes DO
      IF NOT testAxis(axis, v0, v1, v2, e) THEN
         RETURN false
      ENDIF
```

```
    END FOR

    epsilon ← 1e-7

    IF max(v0.x, v1.x, v2.x) < -e.x - epsilon OR
       min(v0.x, v1.x, v2.x) >  e.x + epsilon THEN
        RETURN false
    ENDIF

    IF max(v0.y, v1.y, v2.y) < -e.y - epsilon OR
       min(v0.y, v1.y, v2.y) >  e.y + epsilon THEN
        RETURN false
    ENDIF

    IF max(v0.z, v1.z, v2.z) < -e.z - epsilon OR
       min(v0.z, v1.z, v2.z) >  e.z + epsilon THEN
        RETURN false
    ENDIF

    triNormal ← cross(f0, f1)
    IF NOT testAxis(triNormal, v0, v1, v2, e) THEN
        RETURN false
    ENDIF

    RETURN true
END FUNCTION
```

octree.hpp

```
CONSTANT   TopLeftFront = 0
CONSTANT  TopRightFront = 1
CONSTANT   BottomRightFront = 2
CONSTANT   BottomLeftFront = 3
CONSTANT    TopLeftBottom = 4
CONSTANT   TopRightBottom = 5
CONSTANT    BottomRightBack = 6
CONSTANT    BottomLeftBack = 7


STRUCT Point
    x, y, z : double
END STRUCT


CLASS Octree
    STATIC mutex stats_mutex
    STATIC map node_at_depth
```

```
STATIC map skipped_at_depth
STATIC integer max_reached_depth = 0

isLeaf : boolean
depth : integer
topLeftFront : Point
bottomRightBack : Point
children[8] : Octree


CONSTRUCTOR(x1, y1, z1, x2, y2, z2, d)
   topLeftFront = Point(x1, y1, z1)
   bottomRightBack = Point(x2, y2, z2)
   depth = d
   isLeaf = false
   children = array of 8 NULL

   ACQUIRE LOCK stats_mutex
      node_at_depth[depth] += 1
      IF depth > max_reached_depth THEN
         max_reached_depth = depth
      END IF
   RELEASE LOCK
END CONSTRUCTOR


DESTRUCTOR()
   DELETE topLeftFront
   DELETE bottomRightBack
   FOR each child IN children DO
      IF child != NULL THEN DELETE child
   END FOR
END DESTRUCTOR


FUNCTION build(triangles, maxDepth)
   DEFINE box as AABB(topLeftFront, bottomRightBack)

   intersectingTris = EMPTY LIST

   FOR each triangle IN triangles DO
      IF checkIntersect(box, triangle) THEN
         ADD triangle TO intersectingTris
      END IF
   END FOR

   IF intersectingTris is empty THEN
      IF depth < maxDepth THEN
         ACQUIRE LOCK stats_mutex
            skipped_at_depth[depth + 1] += 8
```

```
            RELEASE LOCK
        END IF
        RETURN
    END IF

    IF depth == maxDepth THEN
        isLeaf = true
        RETURN
    END IF

    COMPUTE midx, midy, midz as midpoint of current box

    CREATE 8 children octants using subdivided bounding boxes

    IF depth < 4 THEN
        CREATE list of futures
        FOR i = 0 to 7 DO
            LAUNCH async task:
                children[i].build(intersectingTris, maxDepth)
        END FOR

        WAIT for all futures to complete
    ELSE
        FOR i = 0 to 7 DO
            CALL children[i].build(intersectingTris, maxDepth)
        END FOR
    END IF

    FOR i = 0 to 7 DO
        IF children[i] != NULL AND children[i].isLeaf == false THEN
            hasAnyChild = false

            FOR j = 0 to 7 DO
                IF children[i].children[j] != NULL THEN
                    hasAnyChild = true
                    BREAK
                END IF
            END FOR

            IF hasAnyChild == false THEN
                ACQUIRE LOCK stats_mutex
                    node_at_depth[children[i].depth] -= 1
                RELEASE LOCK

                DELETE children[i]
                children[i] = NULL
            END IF
        END IF
    END FOR
END FUNCTION
```

```
    FUNCTION collectVoxels(voxels)
        IF isLeaf == true THEN
            DEFINE box as AABB(topLeftFront, bottomRightBack)
            ADD box TO voxels
        ELSE
            FOR i = 0 to 7 DO
                IF children[i] != NULL THEN
                    CALL children[i].collectVoxels(voxels)
                END IF
            END FOR
        END IF
    END FUNCTION


    FUNCTION printStatistics()
        PRINT "Statistics of octree nodes:"
        FOR d = 1 to max_reached_depth DO
            PRINT d, node_at_depth[d]
        END FOR

        PRINT "Skipped nodes:"
        FOR d = 1 to max_reached_depth DO
            PRINT d, skipped_at_depth[d]
        END FOR
    END FUNCTION

END CLASS
```

voxelgenerator.cpp

```
STRUCT Vertex
    x, y, z
END STRUCT

STRUCT Face
    v1, v2, v3
END STRUCT


FUNCTION parseOBJ(path, vertices, faces)
    OPEN file at path
    IF file cannot be opened THEN
        PRINT error
        RETURN
    ENDIF
```

```
   FOR each line in file DO
      READ type

      IF type == "v" THEN
         READ x, y, z
         ADD Vertex(x, y, z) to vertices

      ELSE IF type == "f" THEN
         READ s1, s2, s3

         FUNCTION getIndex(str)
            RETURN integer before '/'
         END FUNCTION

         f.v1 ← getIndex(s1)
         f.v2 ← getIndex(s2)
         f.v3 ← getIndex(s3)

         ADD f to faces
      ENDIF
   END FOR

   CLOSE file
END FUNCTION


FUNCTION getVoxelMinMax(vertices, minX, minY, minZ, maxX, maxY, maxZ)
   IF vertices is empty THEN RETURN

   minX ← maxX ← vertices[0].x
   minY ← maxY ← vertices[0].y
   minZ ← maxZ ← vertices[0].z

   FOR each vertex v DO
      minX ← min(minX, v.x)
      maxX ← max(maxX, v.x)

      minY ← min(minY, v.y)
      maxY ← max(maxY, v.y)

      minZ ← min(minZ, v.z)
      maxZ ← max(maxZ, v.z)
   END FOR
END FUNCTION


FUNCTION writeVoxelOBJ(path, voxels)
   OPEN output file
```

```
    vertexOffset ← 1

    FOR each voxel box DO
        WRITE 8 vertices of cube

        DEFINE 12 triangle faces of cube

        FOR each face DO
            WRITE face with vertexOffset
        END FOR

        vertexOffset ← vertexOffset + 8
    END FOR

    CLOSE file
END FUNCTION

FUNCTION main(argc, argv)

    IF argc < 2 THEN
        PRINT "Usage: <program> <input.obj> [output.obj] [depth]"
        RETURN
    END IF

    inputPath  = argv[1]
    outputPath = (argc > 2) ? argv[2] : default_output_path
    maxDepth   = (argc > 3) ? convert argv[3] to integer : 5

    INITIALIZE empty list vertices
    INITIALIZE empty list faces

    PRINT "Parsing file:", inputPath
    CALL parseOBJ(inputPath, vertices, faces)

    IF vertices is empty THEN
        RETURN
    END IF

    DECLARE minX, minY, minZ, maxX, maxY, maxZ
    CALL getVoxelMinMax(vertices, minX, minY, minZ, maxX, maxY, maxZ)

    maxSize = MAX(maxX - minX, maxY - minY, maxZ - minZ)

    INITIALIZE empty list triangles
    FOR each face f in faces DO
        CREATE triangle using:
            vertices[f.v1 - 1]
            vertices[f.v2 - 1]
            vertices[f.v3 - 1]
        ADD triangle to triangles
```

```
END FOR

startTime = CURRENT TIME

CREATE Octree tree(minX, minY, minZ, maxX, maxY, maxZ)
CALL tree.build(triangles, maxDepth)

stopTime = CURRENT TIME

INITIALIZE empty list voxels
CALL tree.collectVoxels(voxels)

CALL writeVoxelOBJ(outputPath, voxels)

PRINT output path
PRINT statistics:
   number of voxels
   number of vertices
   number of faces
   octree depth
   execution time

CALL tree.printStatistics()
display = OPEN DISPLAY
IF display is NULL THEN RETURN

width  = 1024
height = 768

CREATE window with given width and height
REGISTER input events (keyboard, mouse, motion)
SHOW window

CREATE frameBuffer of size width * height

CREATE renderer(width, height)
SET renderer.rotationY = 0.8
SET renderer.rotationX = 0.5

SET renderer.cameraPos.x = (minX + maxX) / 2
SET renderer.cameraPos.y = (minY + maxY) / 2
SET renderer.cameraPos.z = -(maxSize * 5)

CREATE image buffer linked to frameBuffer
CREATE graphics context

running = true
isRightDragging = false
lastX = 0
lastY = 0
```

```
PRINT "Visualization instructions"
WHILE running DO

    CALL renderer.render(voxels, frameBuffer)
    DRAW frameBuffer to window

    WHILE there are pending events DO
        event = GET next event

        IF event is key press AND key == ESC THEN
            running = false
        END IF

        IF event is mouse button press THEN
            IF right click THEN
                isRightDragging = true
                lastX = mouseX
                lastY = mouseY
            END IF

            IF scroll up THEN
                MOVE camera closer
            END IF

            IF scroll down THEN
                MOVE camera farther
            END IF
        END IF

        IF event is mouse button release AND right click THEN
            isRightDragging = false
        END IF

        IF event is mouse motion AND isRightDragging THEN
            dx = currentX - lastX
            dy = currentY - lastY

            UPDATE renderer.rotationY using dx
            UPDATE renderer.rotationX using dy

            CLAMP rotationX within valid range

            UPDATE lastX, lastY
        END IF

    END WHILE

END WHILE
```

```
    CLOSE display

END FUNCTION
```

Algoritma Divide & Conquer

Program *voxelization* dimulai dengan mencari batas ukuran terkecil dan terbesar untuk sumbu X, Y, dan Z dari keseluruhan objek. Hal ini dilakukan untuk mendefinisikan ruang atau kotak utama (*bounding box*) yang membungkus objek tersebut.

Setelah *bounding box* utama dibuat, tahap divide dijalankan. Pada tahap ini, program mengecek semua segitiga penyusun objek di dalam ruang. Untuk mengetahui apakah sebuah segitiga dan kotak saling berpotongan, program mengujinya dengan menggunakan algoritma *Separating Axis Theorem* (SAT) yang memproyeksikan segitiga dan kotak ke 13 sumbu, di mana kedua bangun tersebut dipastikan tidak bersinggungan jika salah satu sumbu yang hasil proyeksinya tidak saling tumpang tindih. Jika berpotongan, segitiga tersebut akan dimasukkan ke dalam daftar irisan kotak tersebut. Selanjutnya, kotak induk akan dibelah atau *divide* menjadi 8 kotak anak yang lebih kecil. Proses ini berjalan secara rekursif ke dalam masing-masing kotak anak.

Tahap Conquer bekerja sebagai kondisi basis yang akan menghentikan rekursi pada fungsi build tersebut. Pembelahan akan berhenti pada kondisi, dimana tidak terdapat satu pun segitiga yang berpotongan dengan kotak. Jika ini terjadi, program melakukan *pruning* terhadap kotak tersebut dan kotak tidak akan menjadi *voxel*. Kondisi kedua, rekursi berhenti jika *depth* pembelahan telah mencapai batas maksimal, yaitu maxDepth. Jika kotak di maxDepth ini masih memiliki irisan segitiga, maka kotak tersebut akan menjadi satu *voxel*.

Terakhir, tahap Combine pada fungsi build ini memiliki tugas untuk merapikan dan mengumpulkan hasil. Saat proses rekursi bergerak naik kembali, program akan menghapus seluruh memori dari node *octree* yang jalurnya kosong (tidak berujung pada daun/*voxel*). Setelah oc*tree* bersih, fungsi collectVoxels akan dijalankan untuk menelusuri ulang oc*tree* dan menyatukan koordinat semua voxel ke dalam struktur data bernama voxels.

Penjelasan Bonus

Bonus Concurrency

Untuk mendapatkan *concurrency* dalam C++, algoritma ini menggunakan *task-based parallelism* melalui library `<future>` dan `<mutex>`. Eksekusinya beroperasi melalui tiga mekanisme kontrol:

1. Asynchronous Execution (`std::async`):

   Program kami menggunakan `std::async` dengan `std::launch::async` saat membagi node Octree. Perintah ini menginstruksikan *scheduler* sistem operasi untuk mengalokasikan *thread* baru secara langsung. Kedelapan pemanggilan fungsi `build` untuk sub-kubus dikirim ke *thread* terpisah agar kalkulasi interseksi (checkIntersect()) dieksekusi secara paralel pada *core* CPU yang berbeda.

2. Thread Throttling & Barrier Synchronization:

   Penciptaan *thread* baru dibatasi secara eksplisit melalui kondisi depth < 4. Ini dilakukan untuk mencegah terjadinya *thread explosion* dan *overhead context-switching* tingkat perangkat keras dengan membatasi total *thread* maksimum, sementara level kedalaman sisanya dipaksa berjalan sekuensial. Program ini kemudian memanggil `f.get()` pada array objek `std::future` sebagai *synchronization barrier*, yang menahan siklus eksekusi *thread* pemanggil (induk) hingga seluruh *thread* anak mengembalikan status selesai.

3. Mutual Exclusion (`std::mutex`):

   Variabel `node_at_depth` dan `skipped_at_depth` bersifat statis dan diakses memori fisiknya oleh berbagai *thread* sekaligus, sehingga program ini membungkus modifikasinya dengan objek `std::lock_guard<std::mutex>`. Ini mengkuncikan pada tingkat *compiler* yang memastikan hanya satu *thread* yang dapat melakukan instruksi tulis (W) pada blok memori tersebut pada satu siklus *clock*, mengeliminasi *race condition* tanpa memblokir pembacaan data (R) pada variabel lokal lainnya.

Bonus Visualisasi

Sistem visualisasi dibangun menggunakan metode software rasterization dimana seluruh proses kalkulasi dilakukan sepenuhnya oleh CPU. Program berinteraksi dengan sistem operasi dengan menggunakan library X11 (Xlib) untuk manajemen *window* dan pemetaan memori piksel. Program ini terdiri dari beberapa bagian sehingga dapat bekerja :

1. Transformasi Geometri (Rotasi)

   Titik-titik voxel diputar berdasarkan input dari perangkat mouse dengan menggunakan rumus rotasi Euler terhadap titik pusat objek.

   Rotasi terhadap sumbu Y:

$$x_1 = x_0 \cos\theta + z_0 \sin\theta$$

$$z_1 = -x_0 \sin\theta + z_0 \cos\theta$$

Rotasi terhadap sumbu X :

$$y1 = y_0 cos\,\phi - z_1\, sin\,\phi$$

$$z_2 = y_0 sin\,\phi + z_1\, cos\,\phi$$

Keterangan:

θ = sudut rotasi horizontal (azimuth).

ϕ = sudut rotasi vertikal (elevation).

2. Proyeksi Perspektif

Gambar 3D dapat dipetakan menjadi gambar 2D ke layar dengan menggunakan rumus proyeksi perspektif untuk mensimulasikan efek kedalaman.

$$ScreenX = \frac{x1 \cdot f}{z\,final} + \frac{lebar}{2}$$

$$ScreenY = \frac{-y1 \cdot f}{z\,final} + \frac{tinggi}{2}$$

Variabel $z\,final$ ditentukan berdasarkan jarak antara koordinat hasil rotasi dengan posisi kamera:

$$z\,final = z_2 - CameraPos_z$$

3. Manajemen Kedalaman

Tanpa hardware z-buffer, penanganan kedalaman dilakukan dengan Painter's Algorithm. Seluruh wajah (faces) diurutkan berdasarkan nilai Z rata-rata:

$$z\,avg = \frac{za + zb + zc + zd}{4}$$

Wajah digambar dari urutan terjauh ke terdekat agar permukaan depan menutupi permukaan belakang secara akurat.

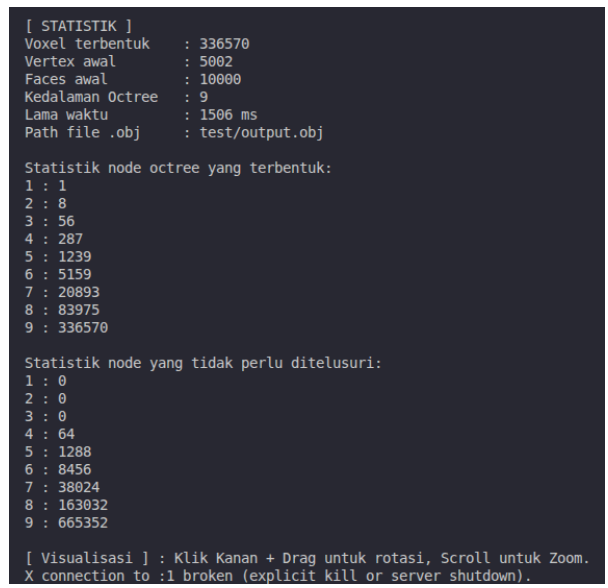
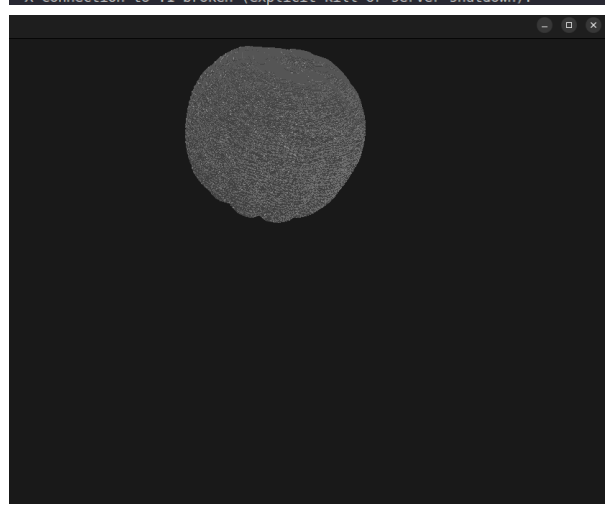
4. Rasterisasi dan Display

Koordinat proyeksi diproses oleh fungsi drawSolidQuad untuk mengisi nilai warna 32-bit ke dalam framebuffer berupa array linear (std::vector<uint32_t>). Framebuffer ditransfer dari RAM ke layar melalui fungsi low-level XPutImage pada protokol X11.

Lampiran

Link Github : https://github.com/N8NAS/Tucil2_13524041_13524130.git

Tampilan Input dan Output dari program

Input	Output
./bin/voxel_gen test/pumpkin.obj test/output.obj 9	 

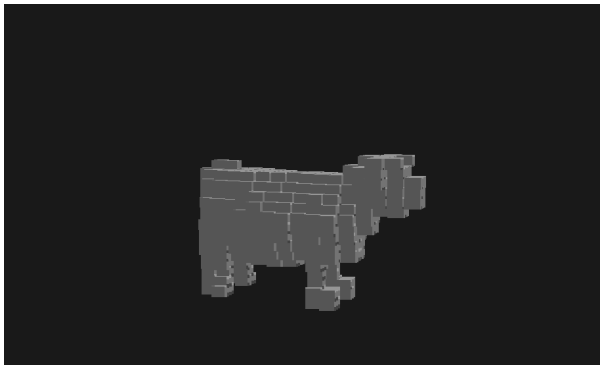
./bin/voxel_gen test/cow.obj test/output.obj 6

```
[Parsing file obj] : test/cow.obj
[Output di path  ] : test/output.obj


[ STATISTIK ]
Voxel terbentuk    : 3984
Vertex awal        : 4583
Faces awal         : 5804
Kedalaman Octree   : 6
Lama waktu         : 34 ms
Path file .obj     : test/output.obj

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 48
4 : 236
5 : 944
6 : 3897

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 0
3 : 0
4 : 128
5 : 1184
6 : 7488
```



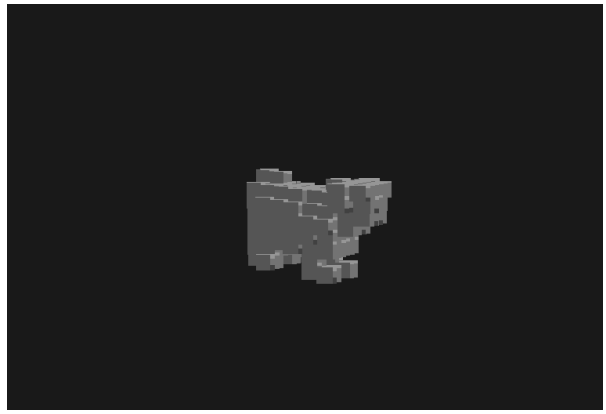
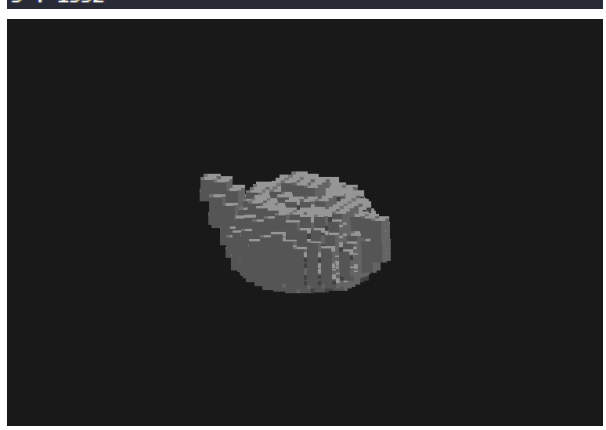
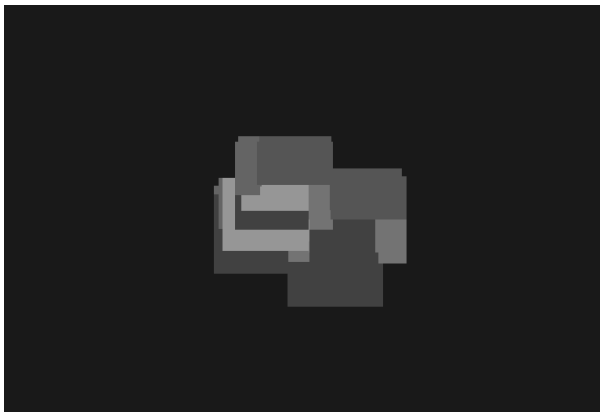
./bin/voxel_gen test/cow.obj test/output.obj 4

```
[Parsing file obj] : test/cow.obj
[Output di path  ] : test/output.obj


[ STATISTIK ]
Voxel terbentuk    : 236
Vertex awal        : 4583
Faces awal         : 5804
Kedalaman Octree   : 4
Lama waktu         : 24 ms
Path file .obj     : test/output.obj

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 48
4 : 231

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 0
3 : 0
4 : 128
```

	
./bin/voxel_gen test/teapot.obj test/output.obj	[Parsing file obj] : test/teapot.obj [Output di path] : test/output.obj [STATISTIK] Voxel terbentuk : 938 Vertex awal : 530 Faces awal : 1024 Kedalaman Octree : 5 Lama waktu : 99 ms Path file .obj : test/output.obj Statistik node octree yang terbentuk: 1 : 1 2 : 8 3 : 52 4 : 222 5 : 940 Statistik node yang tidak perlu ditelusuri: 1 : 0 2 : 0 3 : 0 4 : 96 5 : 1552 

./bin/voxel_gen test/teapot.obj test/output.obj 2	[Parsing file obj] : test/teapot.obj [Output di path] : test/output.obj [STATISTIK] Voxel terbentuk : 8 Vertex awal : 530 Faces awal : 1024 Kedalaman Octree : 2 Lama waktu : 2 ms Path file .obj : test/output.obj Statistik node octree yang terbentuk: 1 : 1 2 : 8 Statistik node yang tidak perlu ditelusuri: 1 : 0 2 : 0